Practical No.1

Aim: Implementing Substitution and Transposition Ciphers: Design and implement algorithms to encrypt and decrypt messages using classical substitution and transposition techniques.

Code for implementing Substitution Cipher with Caesar Cipher:

```
def caesar_cipher(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            offset = ord('a') if char.islower() else ord('A')
            result += chr((ord(char) - offset + shift) % 26 + offset)
        else:
            result += char
    return result


# Example usage
message = input("Enter the text to encrypt: ")
shift = 3

encrypted_message = caesar_cipher(message, shift)
print("Encrypted:", encrypted_message)

decrypted_message = caesar_cipher(encrypted_message, -shift)
print("Decrypted:", decrypted_message)
```

Output :

```
======================== RESTART: E:\INS\Ceasercipher.py
Enter the text to encrypt: PRIYA RAJAK
Encrypted: SULBD UDMDN
Decrypted: PRIYA RAJAK
```

SFCM 2025-26

Code for implementing transposition Cipher using Railfence Cipher :

```
def rail_fence_encrypt(text):
    return text[::2] + text[1::2]


def rail_fence_decrypt(cipher):
    half = (len(cipher) + 1) // 2
    first, second = cipher[:half], cipher[half:]
    return ''.join(
        first[i // 2] if i % 2 == 0 else second[i // 2]
        for i in range(len(cipher))
    )


# Main
if __name__ == "__main__":
    text = input("Enter the text: ")

    encrypted = rail_fence_encrypt(text)
    print("Encrypted:", encrypted)

    decrypted = rail_fence_decrypt(encrypted)
    print("Decrypted:", decrypted)
```

Output:

```
======================= RESTART: E:\INS\Raifencecipher.py =
Enter the text: PRIYARAJAK
Encrypted: PIAAARYRJK
Decrypted: PRIYARAJAK
```

SDSIT 2025-26

Practical No.2

Aim: RSA Encryption and Decryption: Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
import binascii

# Generate a new RSA key pair
keyPair = RSA.generate(1024)

# Extract the public key
pubKey = keyPair.publickey()
print(f"Public key: (n={hex(pubKey.n)}, e={hex(pubKey.e)})")

# Export the public key to PEM format
pubKeyPEM = pubKey.exportKey()
print(pubKeyPEM.decode('ascii'))

# Extract the private key
privKey = keyPair
print(f"Private key: (n={hex(privKey.n)}, d={hex(privKey.d)})")

# Export the private key to PEM format
privKeyPEM = privKey.exportKey()
print(privKeyPEM.decode('ascii'))

# Encryption
msg = b'SDSM College'  # Message in bytes
encryptor = PKCS1_OAEP.new(pubKey)
encrypted = encryptor.encrypt(msg)
print("Encrypted:", binascii.hexlify(encrypted))

# Decryption
decryptor = PKCS1_OAEP.new(privKey)
decrypted = decryptor.decrypt(encrypted)
print("Decrypted:", decrypted.decode('utf-8'))
```

SDSM 2025-26

Output:

```
============================== RESTART: E:\INS\RSA.py =============
Public key: (n=0xc319cd7997fad7b735ec01490c6a824d8a3d886d6b5e509f
2c4cc15eee025c963a5cfcc6a4990c531df7f3e1768d8e202424561ed91ebc3af
01)
-----BEGIN PUBLIC KEY-----
MIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQKBgQDDGc15l/rXtzXsAUkMaoJNij2I
bWteUJ/O+pU+phMXUzh3tvzbYZ2uGfgV5CziP4sXEc8r6xFyEufPI+V95yxMwV7u
AlyWOlz8xqSZDFMd9/Phdo2OICQkVh7ZHrw6/H8QP/i/hB48v2pyN+N3G/7bz01q
PcvQDIMUaC2QtU+PMQIDAQAB
-----END PUBLIC KEY-----
Private key: (n=0xc319cd7997fad7b735ec01490c6a824d8a3d886d6b5e509
72c4cc15eee025c963a5cfcc6a4990c531df7f3e1768d8e202424561ed91ebc3a
4b6e29cf3e81abf97fe9f9ca194946b0ae86fc0a393f1244d015b55678bd5c836
e3a8f1e9c38ca2590236568ae79028f28c9db815c32b5e02a8ed1367401122abb
-----BEGIN RSA PRIVATE KEY-----
MIICWgIBAAKBgQDDGc15l/rXtzXsAUkMaoJNij2IbWteUJ/O+pU+phMXUzh3tvzb
YZ2uGfgV5CziP4sXEc8r6xFyEufPI+V95yxMwV7uAlyWOlz8xqSZDFMd9/Phdo2O
ICQkVh7ZHrw6/H8QP/i/hB48v2pyN+N3G/7bz01qPcvQDIMUaC2QtU+PMQIDAQAB
An83S24pzz6Bq/l/6fnKGUlGsK6G/Ao5PxJE0BW1Vni9XINro3EjLfktQmfUsA+W
uqSUrFsLF3f2wEiA+uRlWMo06Z9AknyWNE46jx6cOMolkCNlaK55Ao8oyduBXDK1
4CqO0TZ0ARIqu7mLtO7IgEj85oFomvc3u0zCslBPaDFBAkEA0Lnvc2sy57Q6V4zO
4Y/6nbX9S+gifrNfza+HcIY2tYX+t9953TFtXsUKl4bTTuMth431yPJQTLowLLh6
KDeZuQJBAO9J2jKPNcLFBrvhF16rGXu8RWvXoHfFol/+8aXZPzbxRCmkqtD740cn
XE2FpJ9kDOP/cVI0rxPQkFlh2sVXfTkCQQDPRiW0r1qw37gkIw96zZjprsUn8RD4
UzarMklEg/OmVbc6b3E8KA5dUz0OdnQ23cDj4SSX9ZGqcCbQ8i1U7Mn5AkBTNBbO
5cUHyIKi6Y26uTvPLpPk/8EoZGwlfPyBcpzsec/9uug6apuIrkMGIaYwVhW1YakB
sJgAB9FpgwETBWzhAkAsjBi4JsJM3e3W1qEcw8QXH7XHQIz+PhmBRngRXITkoeAc
0xJl3dVIvxFS8stp1ikmgTEXtg1+Ss9NNoJsvgP3
-----END RSA PRIVATE KEY-----
Encrypted: b'8b4d057f3b80a7d52d0e059eeaebb2c3b89698562d49ca0b5353
ff350a72bb5de321af87779b2f2a73a681505eb859d6acebe2ca274e26c0526ab
Decrypted: SDSM College
```

Practical No.3

Aim: Message Authentication Codes:
Implement algorithms to generate and verify message authentication codes (MACs) for ensuring data integrity and authenticity.

Code for implementing MD5 Algorithm :

```
import hashlib
result = hashlib.md5 (b'Priya')
result1 = hashlib.md5 (b'Rajak')
# printing the equivalent byte value.
print ("The byte equivalent of hash(Priya):", end ="")
print(result.digest ())
print ("The byte equivalent of hash(Rajak) :", end ="")
print (result1.digest () )
```

Output:

```
========== RESTART: E:\INS\MD5.py ====
The byte equivalent of hash(Priya):b'q\xe9\x1c\xbc}\x97u\x8b_a\x92nNh,\x12'
The byte equivalent of hash(Rajak) :b'f\x06\x07\xb7[\n\xe1\x1c\xe6%\xfd\xb4I\xe1\xdep'
```

Code for implementing SHA Algorithm :

```
import hashlib
text = input("Enter the value to encode: ")
result = hashlib.sha1(text.encode())
print("The Hexadecimal Equivalent of SHA1 is:")
print(result.hexdigest())
```

Output:

```
============================ RESTART: E:\INS\SHA.py
Enter the value to encode priyarajak
The Hexadecimal Equivalent if SHA1 is:
aba01ce8a9d5bca1f416cbf0e8a6f4d5b8b3ac2c
```

SDSM 2025-26

Practical No.4

Aim: Digital Signatures: Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

Python code for implementing SHA Algorithm :

```
from Crypto.PublicKey import RSA
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256

# Generate RSA key pair
key = RSA.generate(2048)
private_key = key.export_key()
public_key = key.publickey().export_key()

# Simulated document content
original_document = b"This is the original document content."
modified_document = b"This is the modified document content."

# Hash the document content
original_hash = SHA256.new(original_document)
modified_hash = SHA256.new(modified_document)

# Create a signature using the private key
signature = pkcs1_15.new(RSA.import_key(private_key)).sign(original_hash)

# Verify the signature using the public key with the modified content
try:
    pkcs1_15.new(RSA.import_key(public_key)).verify(modified_hash,
signature)
    print("Signature is valid.")
except (ValueError, TypeError):
    print("Signature is invalid.")
```

Output :

```
======================== RESTART: E:\INS\digitalsign.py
Signature is invalid.
```

SDSM 2025-26

Practical No.5

Aim: Key Exchange using Diffie-Hellman:
Implement the Diffie-Hellman key exchange algorithm to securely exchange keys between two entities over an insecure network.

Code for implementing Diffie-Hellman Algorithm :

```
if __name__ == "__main__":
    N = 23
    G = 37

    print('The Value of N is: %d' % N)
    print('The Value of G is: %d' % G)

    a = 4
    print('Secret Number for Priya is: %d' % a)
    x = pow(G, a, N)  # Priya's public value

    b = 5
    print('Secret Number for Preet: %d' % b)
    y = pow(G, b, N)  # Preet's public value

    ka = pow(y, a, N)  # Priya computes shared key
    kb = pow(x, b, N)  # Preet computes shared key

    print('Secret Key for Priya is: %d' % ka)
    print('Secret Key for Preet is: %d' % kb)
```

Output:

```
========== RESTART: E:\INS\hellman.py =
The Value of N is: 23
The Value of G is: 37
Secret Number for Priya is: 4
Secret Number for Preet: 5
Secret Key for Priya is: 2
Secret Key for Preet is: 2
```

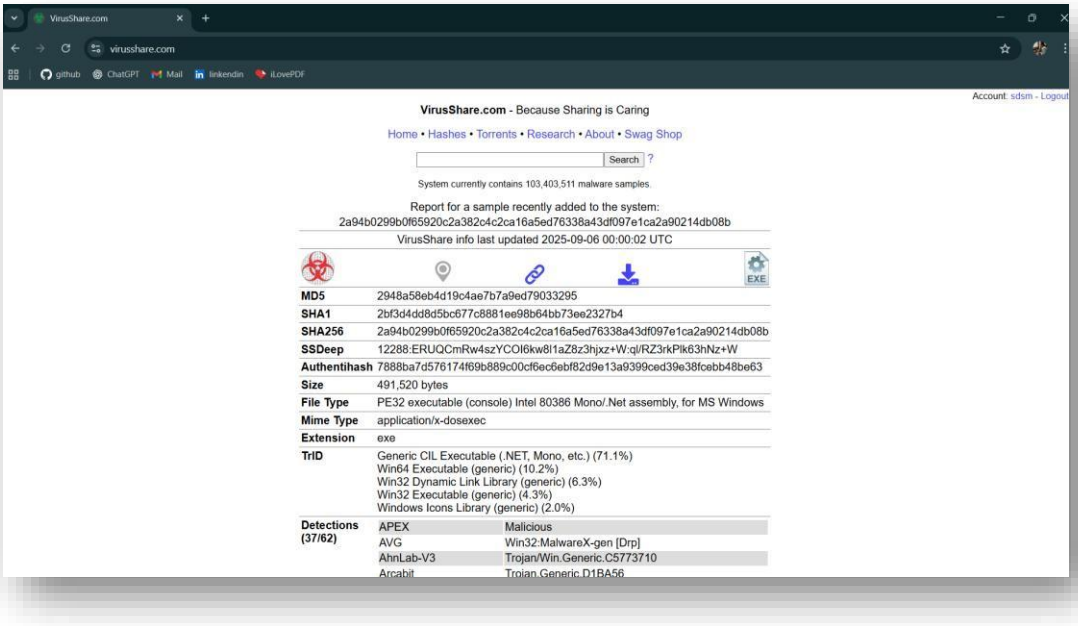
CSM 2025-26

Practical No.6

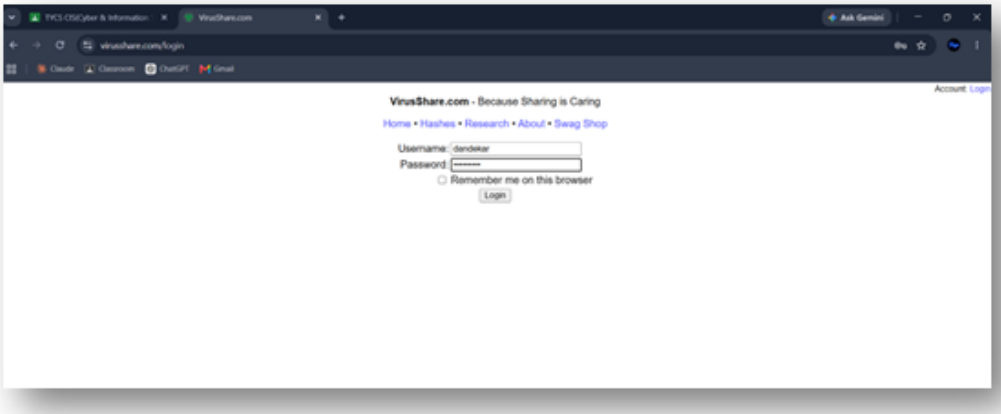
Aim: Malware Analysis and Detection

For analyzing the Malware, we need one. A clean sample of the Malware needs to be downloaded from a trusted website, the downloading and analysis is demonstrated by the following steps:

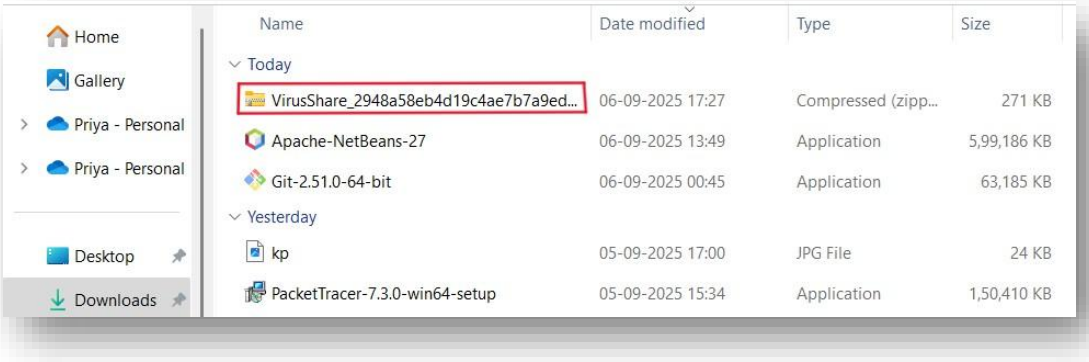
1) We select the website www.virusshare.com for downloading the clean sample of Malware (an account needs to be created for the same). Any other source can be selected to download the Malware (clean sample and authorized site)



2) Open the VirusShare website and click **Account** → **Login**; enter the provided username **"dandekar"** and password **"sdsm@2026"** and click **Login**. (If login credentials are not available, click **About** and follow the instructions to send an email requesting access credentials.)

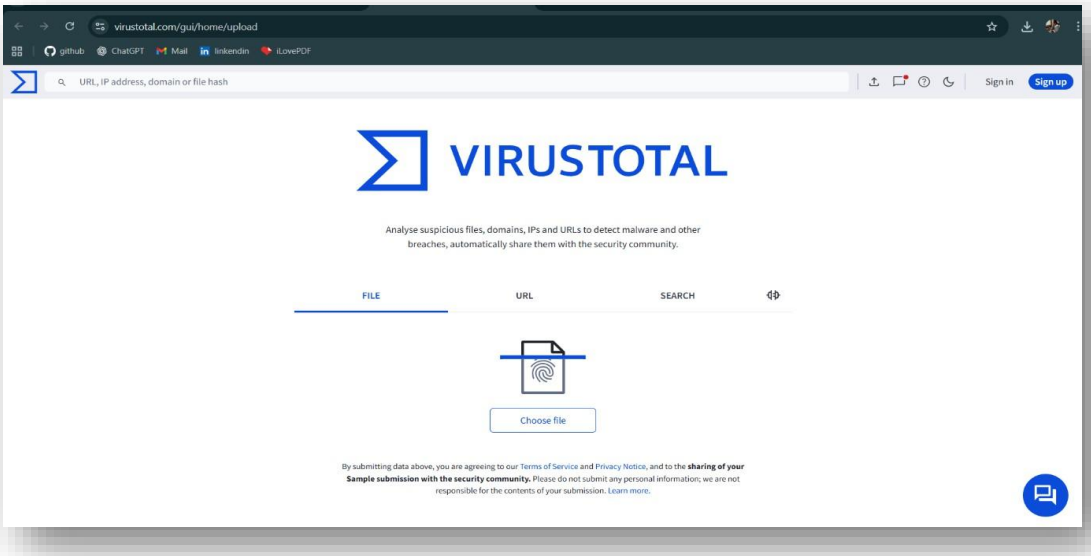


3) By clicking the above download icon the Malware gets downloaded in ZIP format.

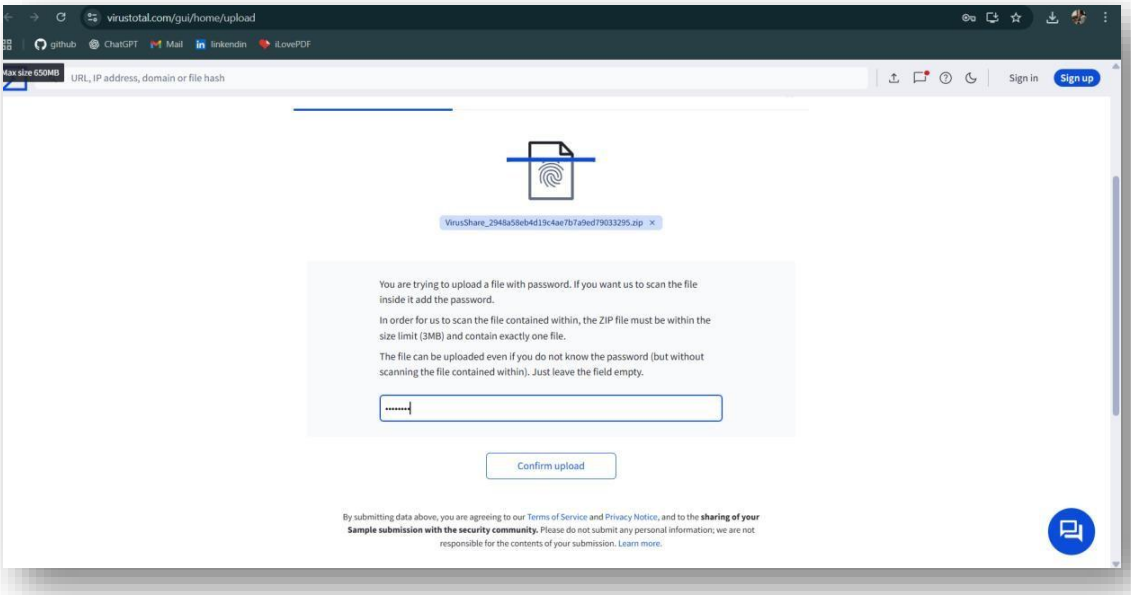


4) For unzip the password is “infected”, there is no need to unzip the file, we create a folder “Malware” on desktop and save the file in the folder

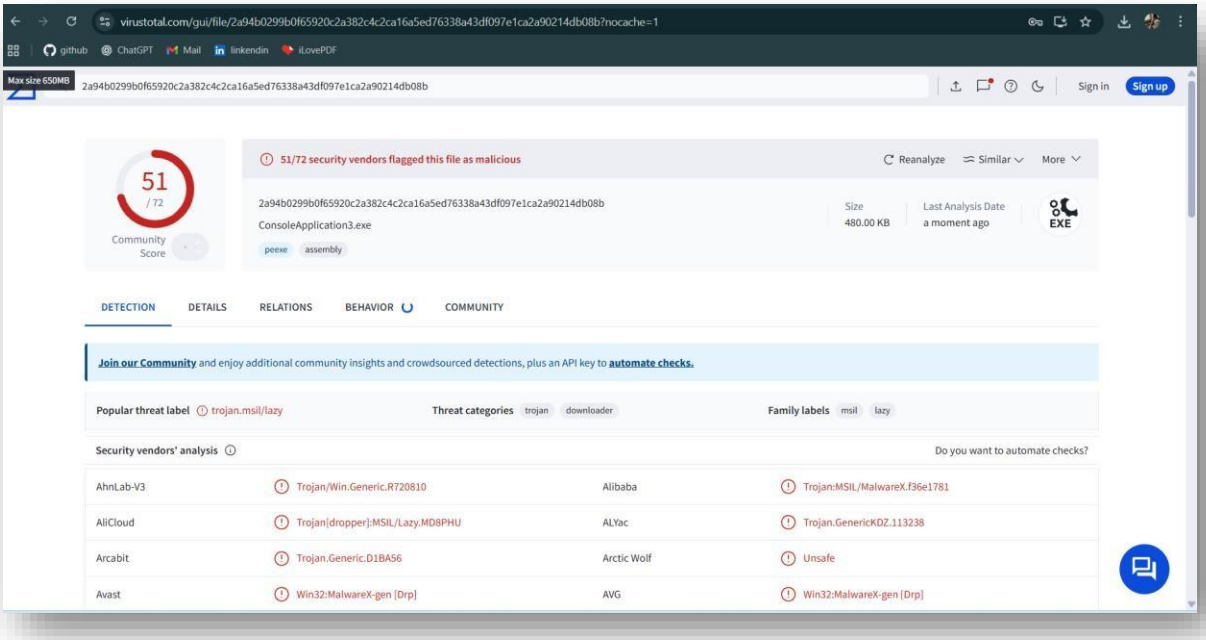
5) In order to analyze the Malware, we select the website www.virustotal.com



6) Click on “Choose File” and select the file from the location (ZIP file will do, if asks for password enter infected)

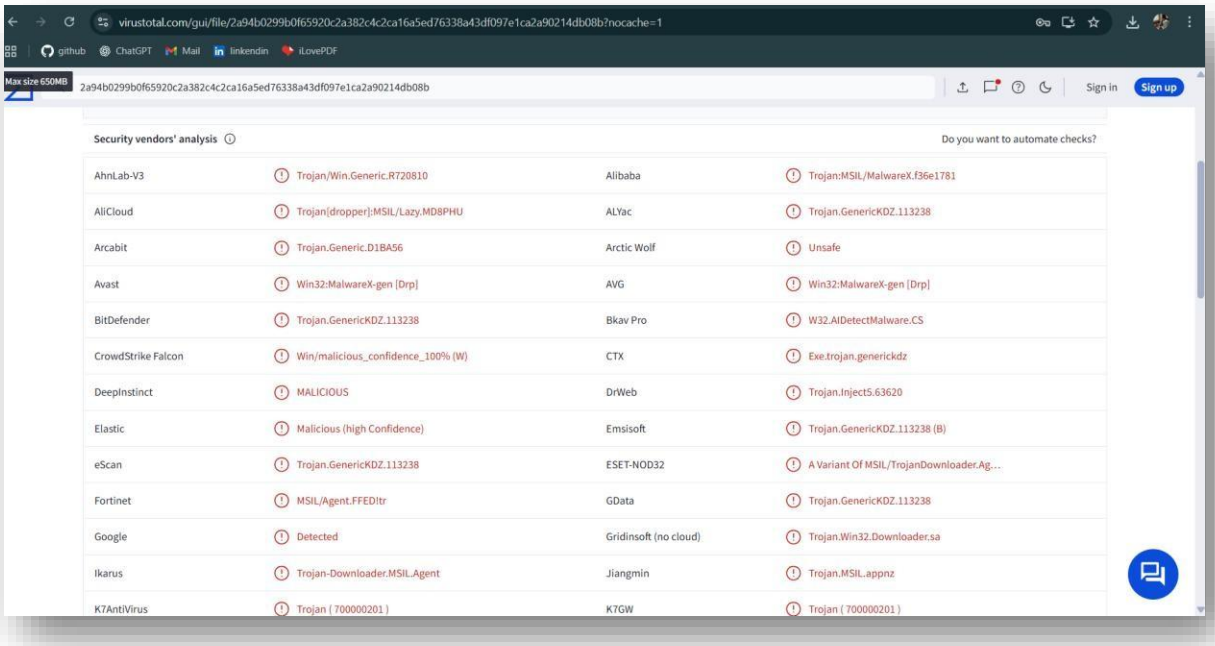


7) We get the following after the upload is complete



We interpret the following findings

a) 51 security vendors out of 51 flagged this file as malicious
The detection tab shows the threats-type

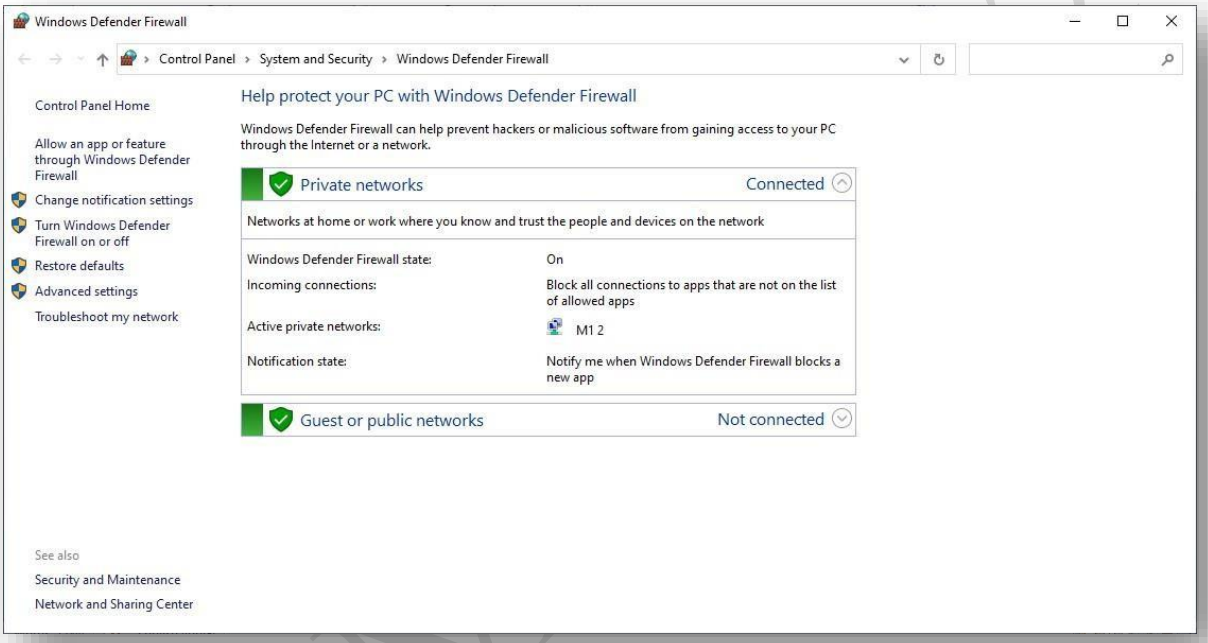


Practical No.8

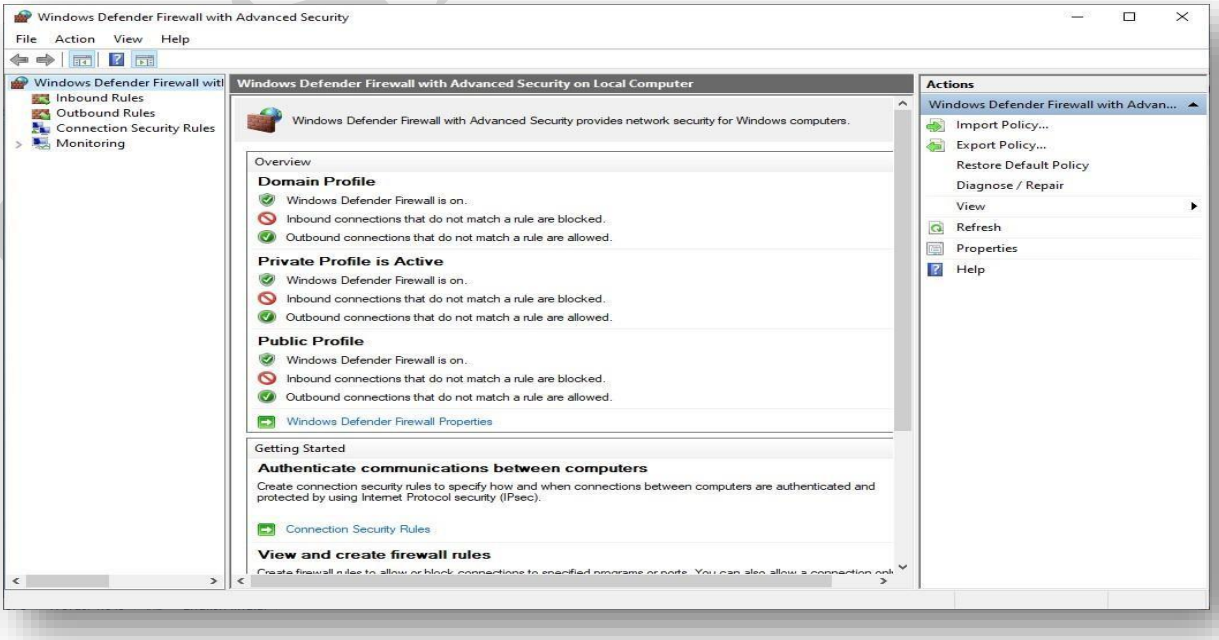
Aim: Firewall Configuration and Rule-based Filtering:
Configure and test firewall rules to control network traffic, filter packets based on specified criteria, and protect network resources from unauthorized access.

Step 1: We access any website through the browser and confirm that the HTTP/HTTPS protocols are working.

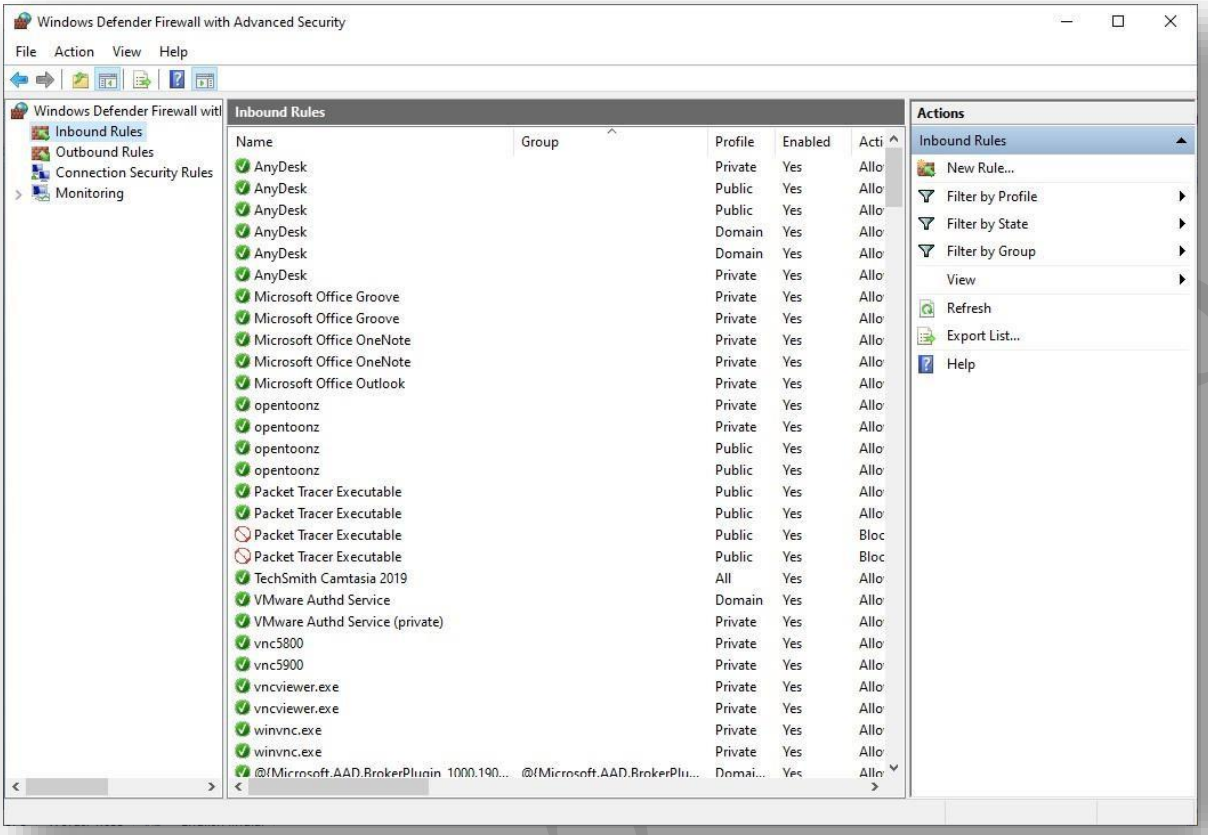
Step 2: We open 'Windows Defender Firewall'



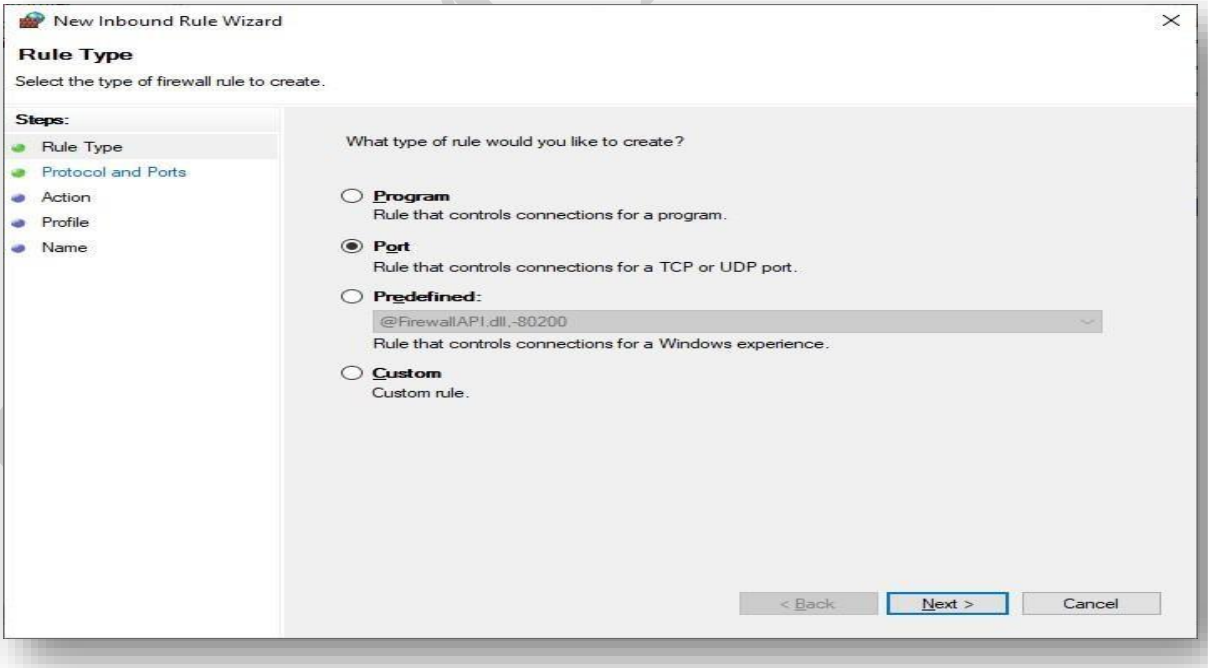
Next we click on 'Advanced settings'



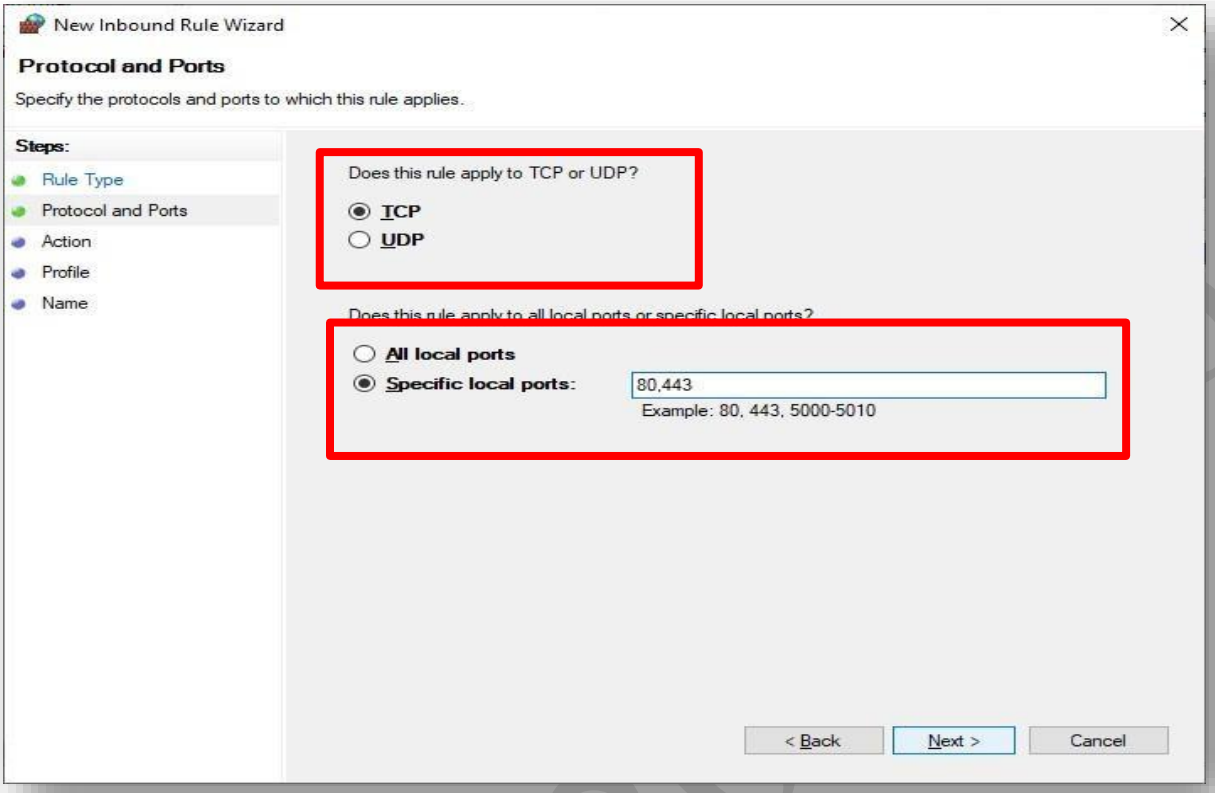
Next we click on ‘Inbound Rules’



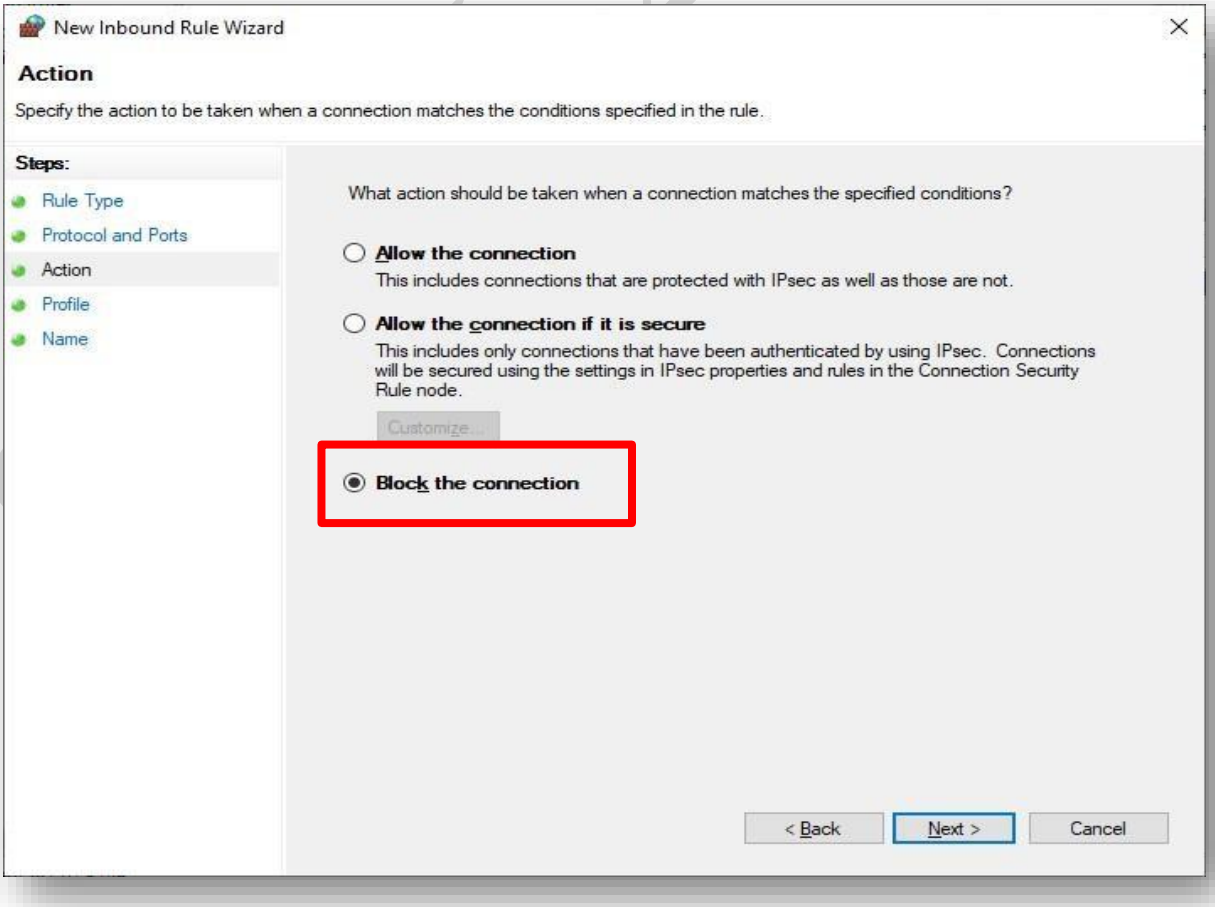
Then click on ‘New Rule’



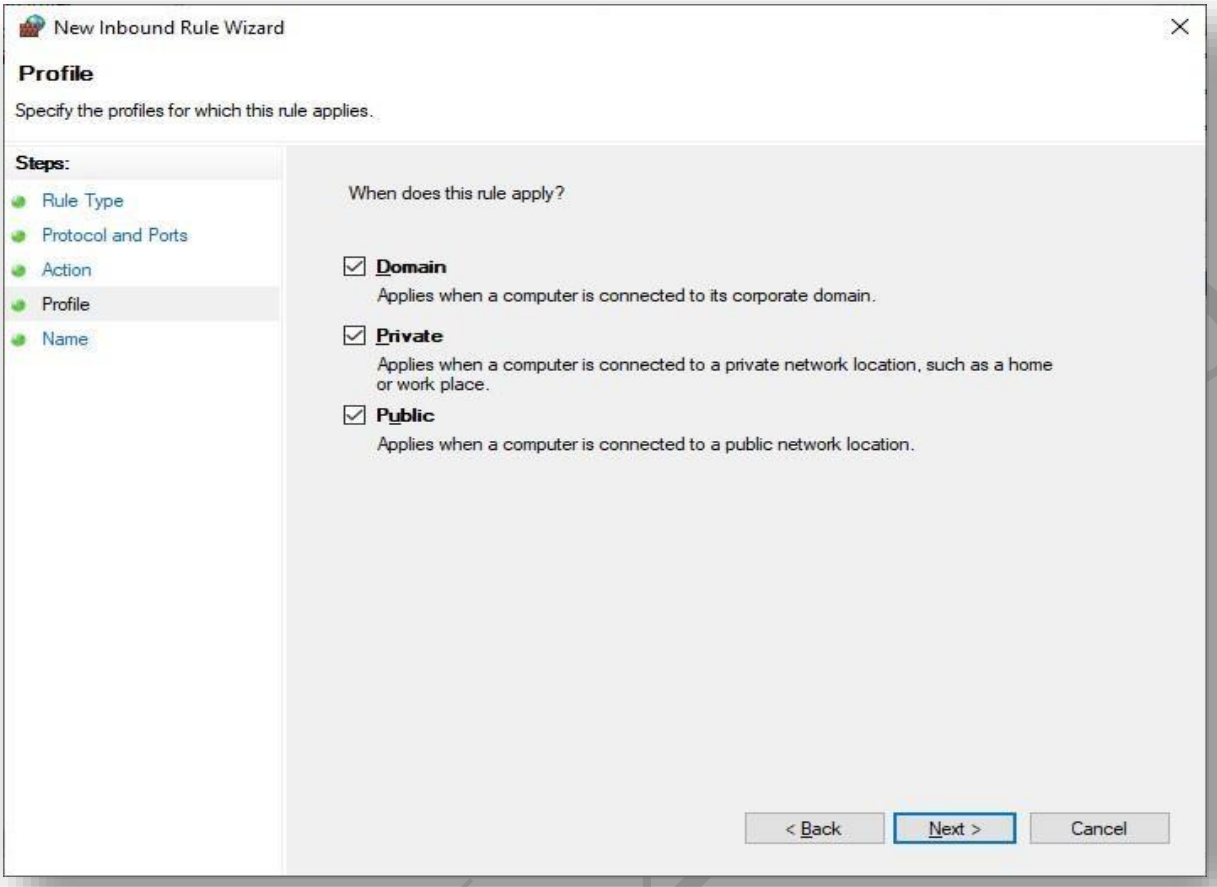
Select the radio button ‘Port’ and click ‘Next’ and enter the following



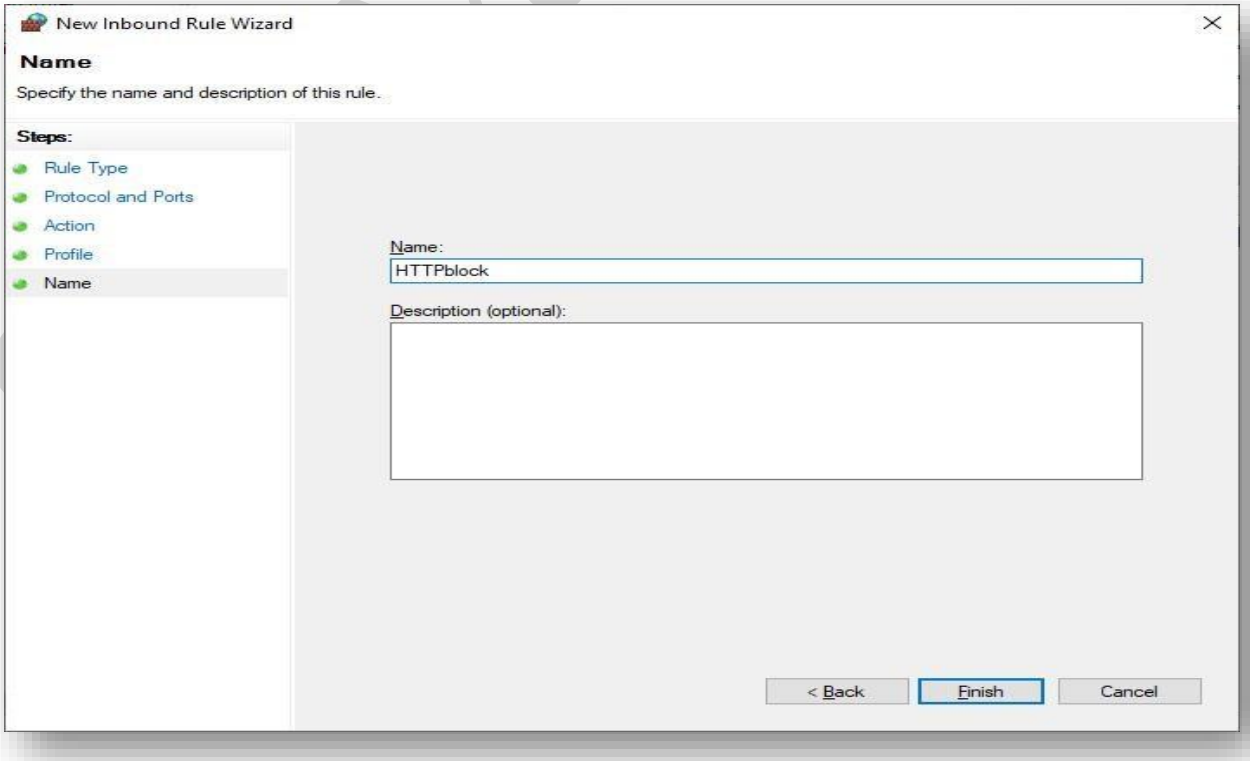
next, we need to finalise the rule



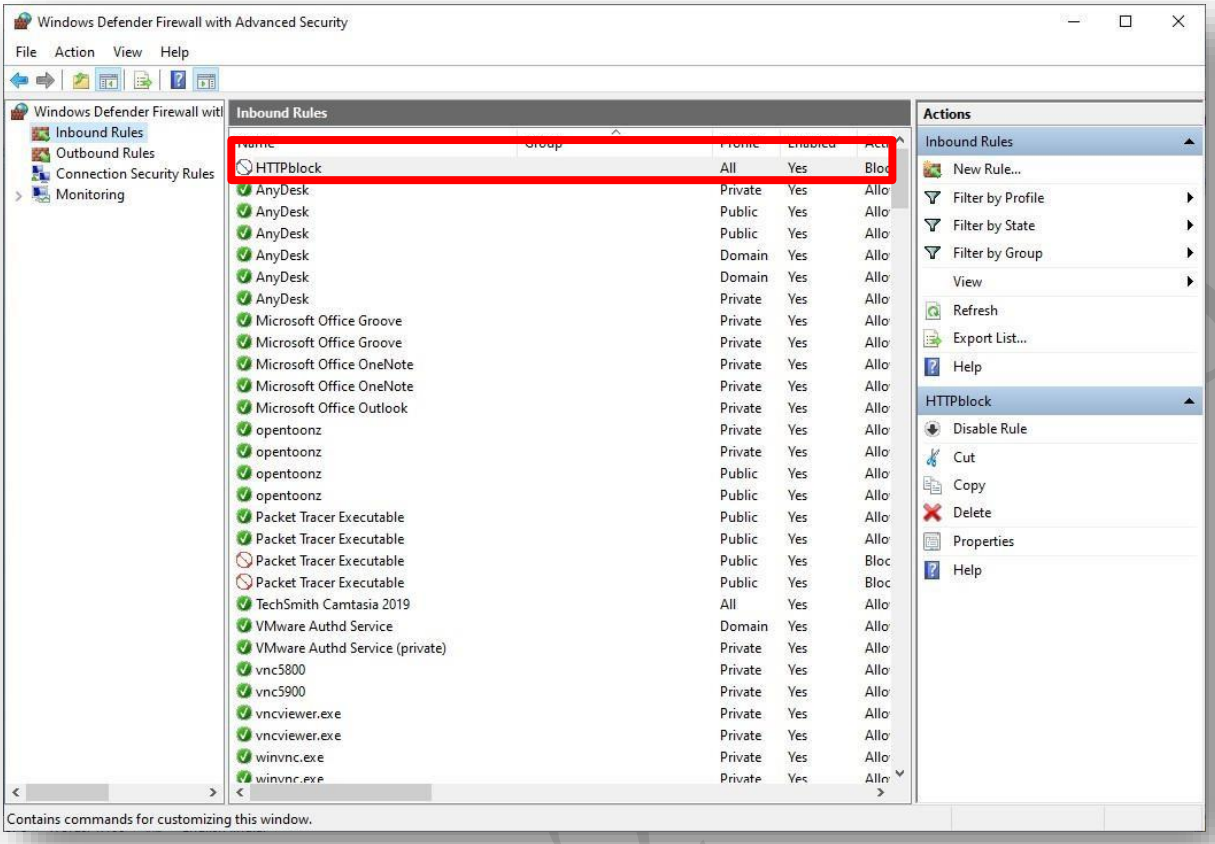
Click ‘Next’ and we get the following



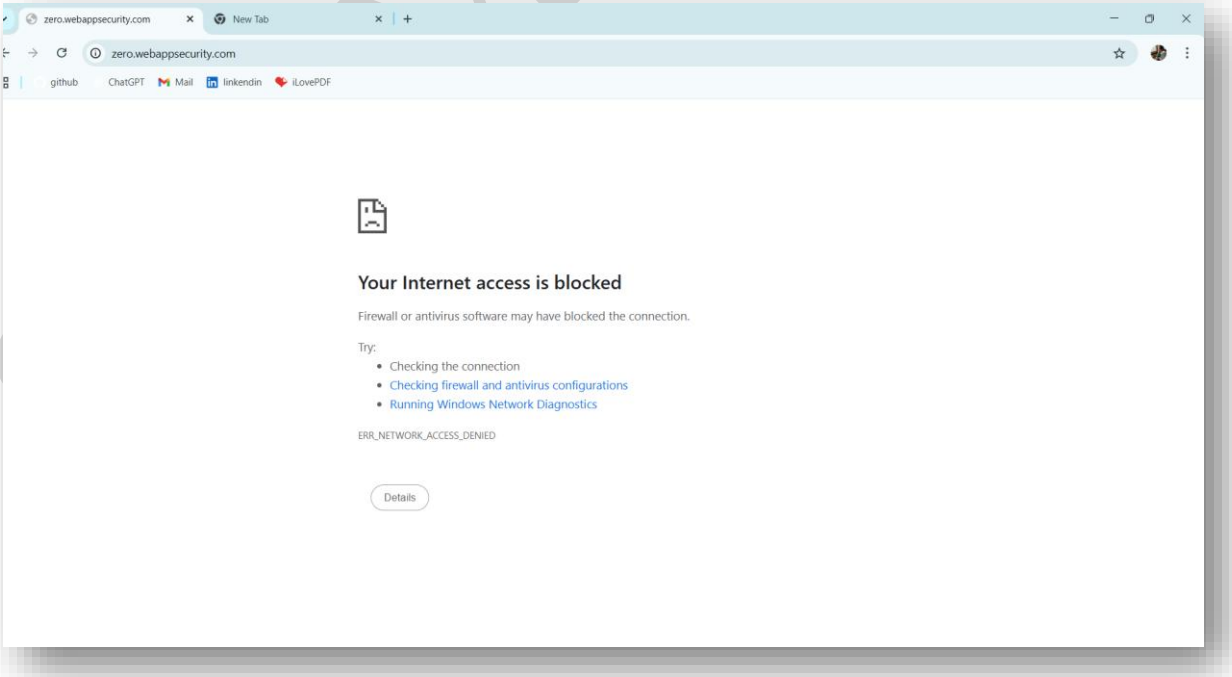
After clicking the ‘Next’ button we need to name the rule and click finish



Inbound rule is added

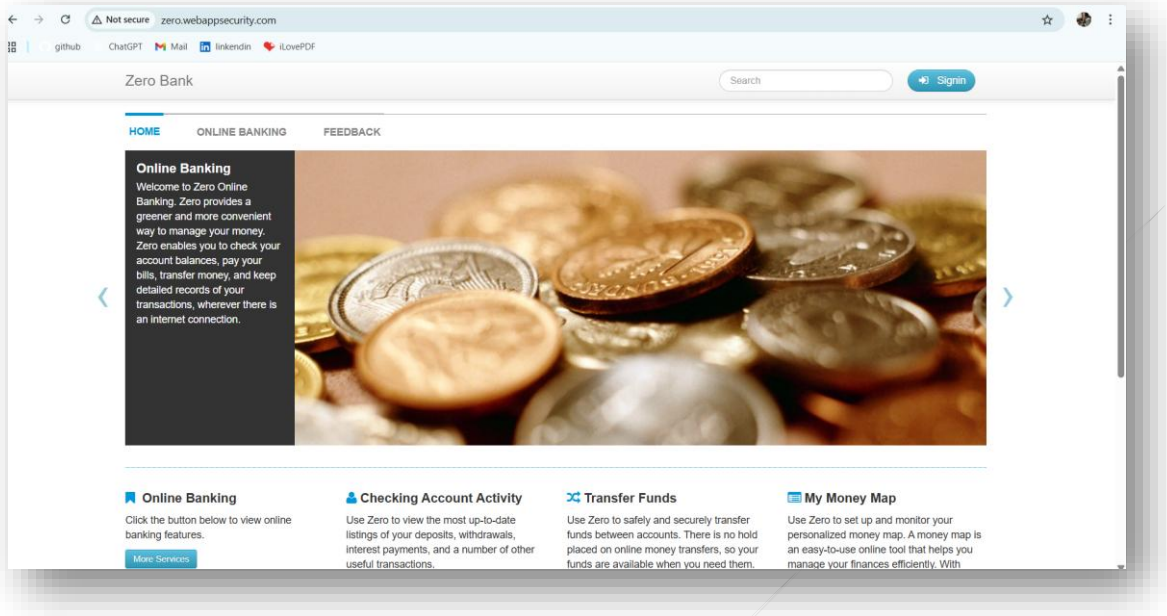


We repeat all the above steps for creating ‘Outbound Rules’, and then try to access the internet. We see that the accessed is blocked

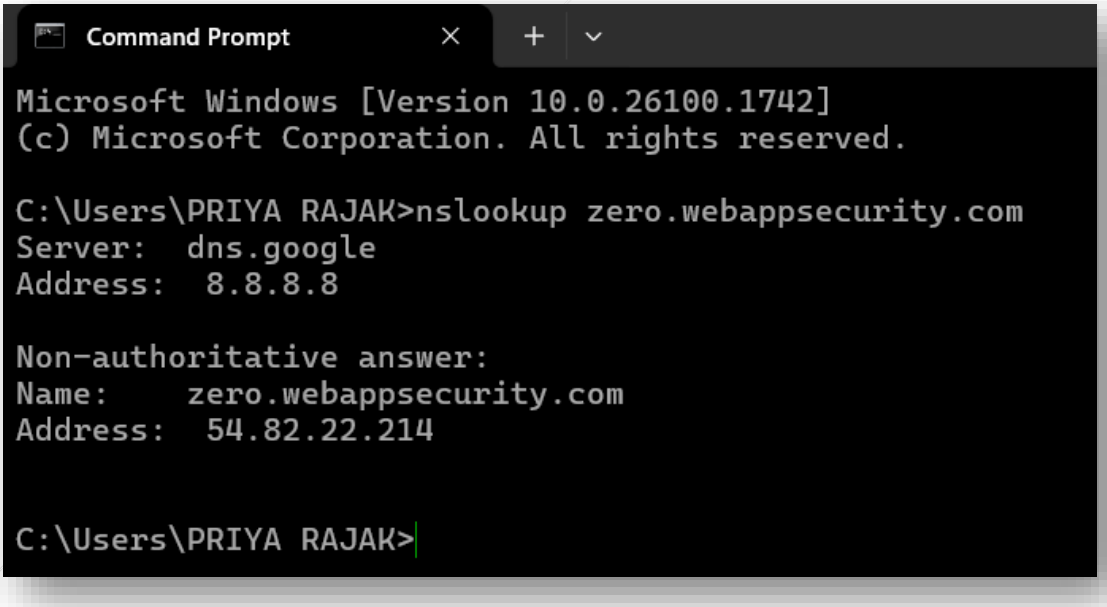


Part 2: Blocking the website zero.webappsecurity.com

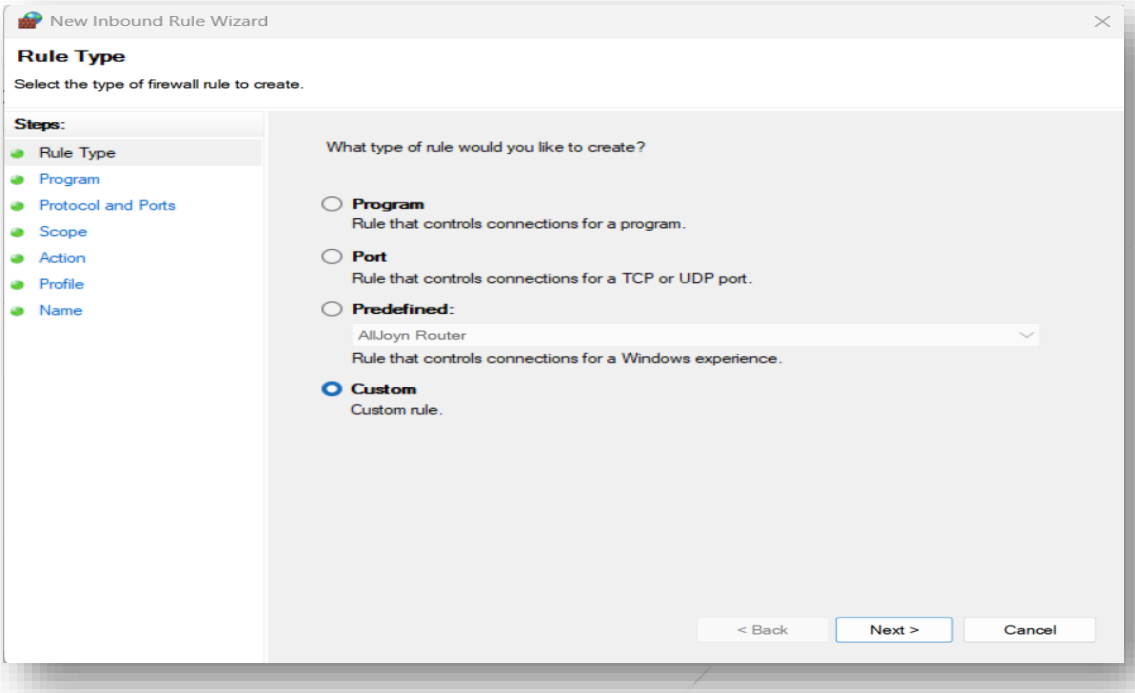
We open the browser and access the website, which is now accessible

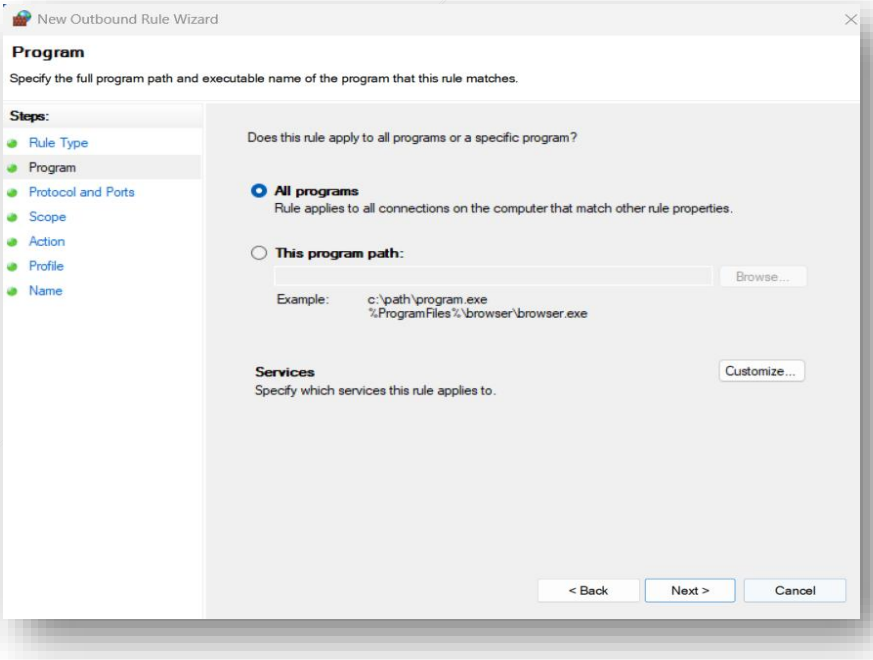


We find the IP addresses of the website using the following command

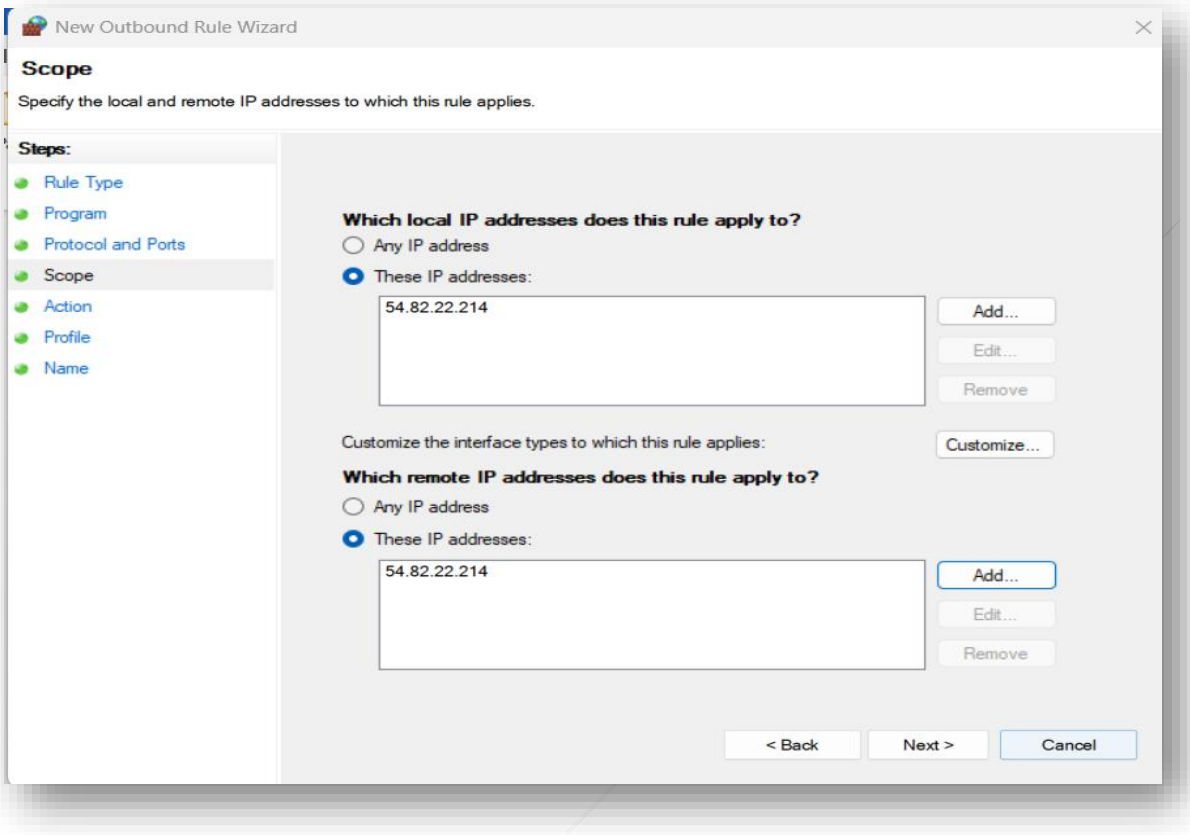


We open the windows Firewall settings and apply the Inbound Rule

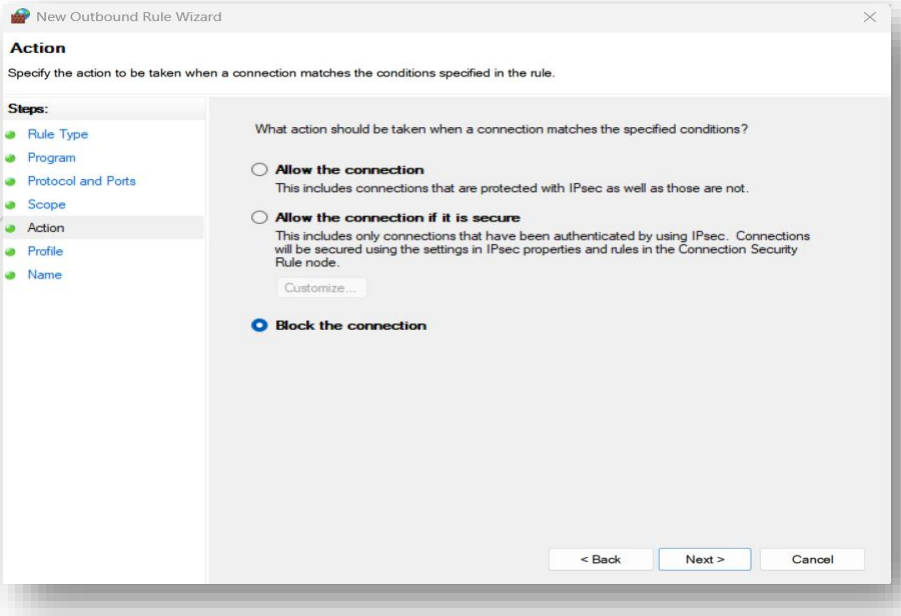




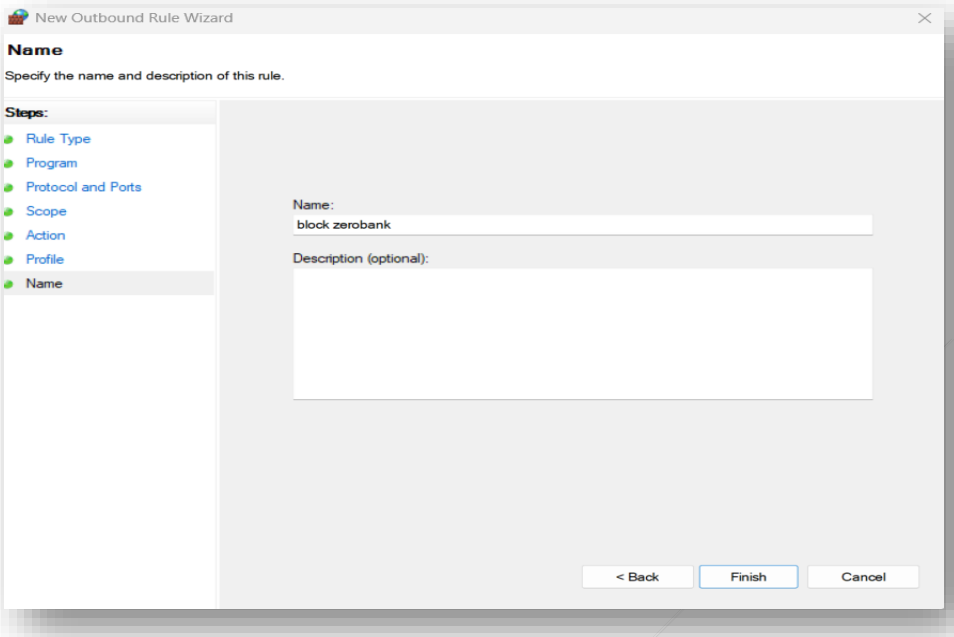
Insert the IP addresses both IPv4 and IPv6



Select Block connection

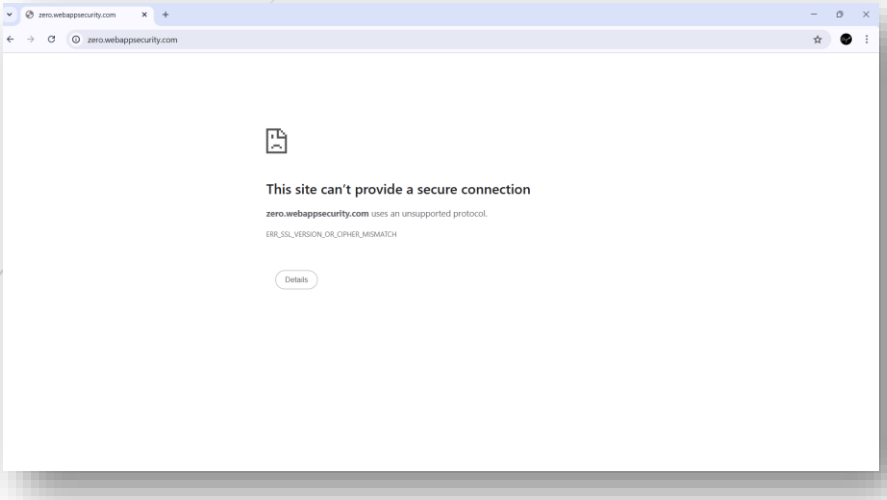


Provide a suitable name and finish



Repeat the above for Outbound Rules

Now if we try to access the website zero.webappsecurity.com , it would be blocked.

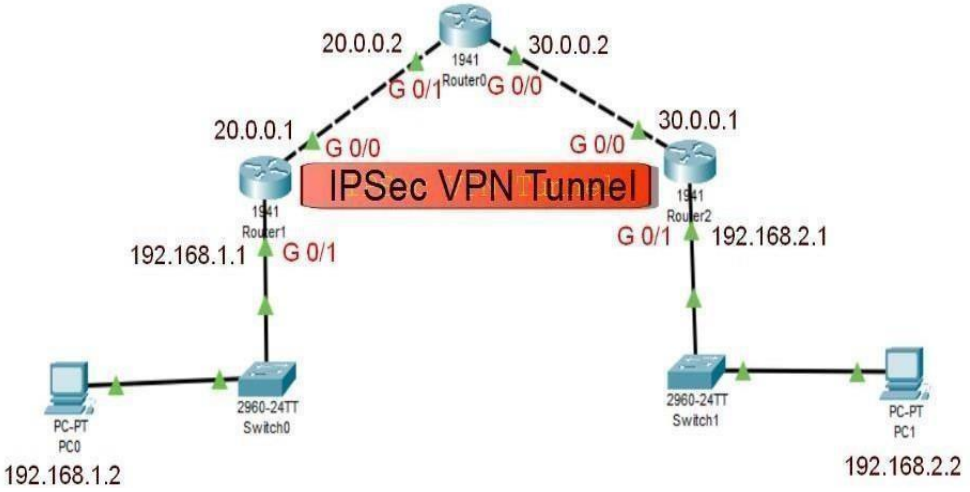


Practical No. 8

Aim: IP Security (IPsec)

Configuration:

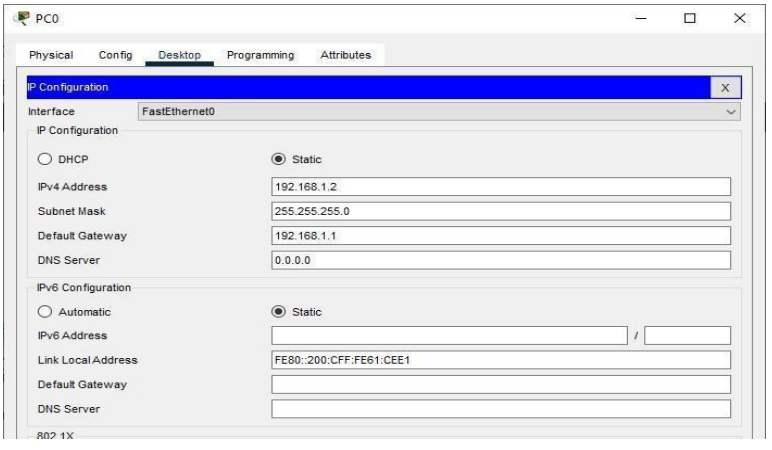
Configure IPsec on network devices to provide secure communication and protect against unauthorized access and attacks.



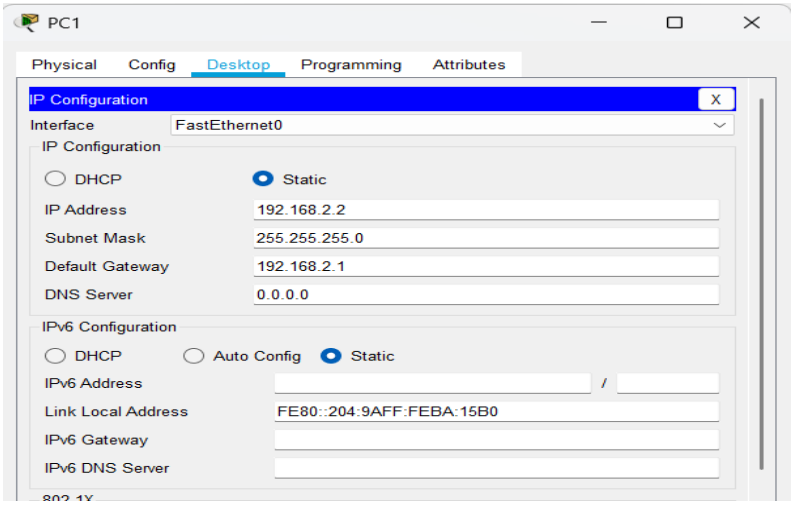
IPSec Network Addressing Table

Device	Interface	IP Address	Subnet Mask	Remarks
R0	g0/0	30.0.0.2	255.255.255.252	Link to R2
R0	g0/1	20.0.0.2	255.255.255.252	Link to R1
R1	g0/0	20.0.0.1	255.255.255.252	Link to R0
R1	g0/1	192.168.1.1	255.255.255.0	LAN (PC1)
R2	g0/0	30.0.0.1	255.255.255.252	Link to R0
R2	g0/1	192.168.2.1	255.255.255.0	LAN (PC2)
PC1	NIC	192.168.1.2	255.255.255.0	Connected to R1
PC2	NIC	192.168.2.2	255.255.255.0	Connected to R2

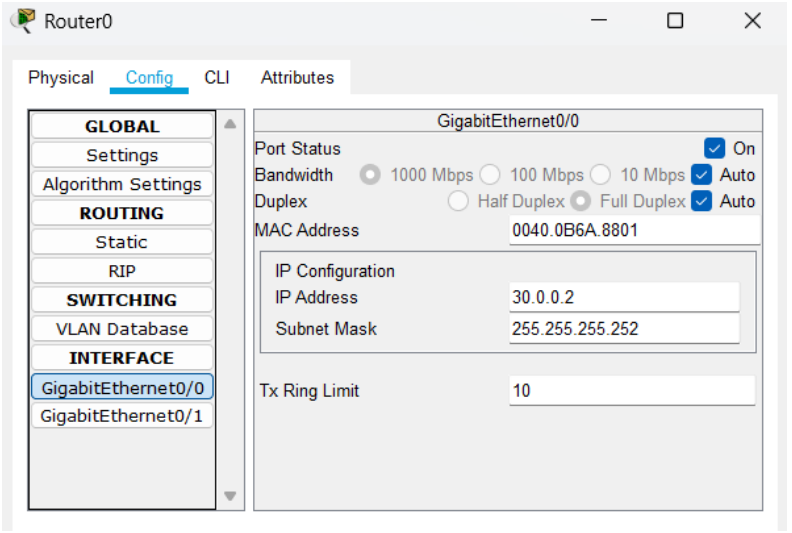
1. Configuring PC0:



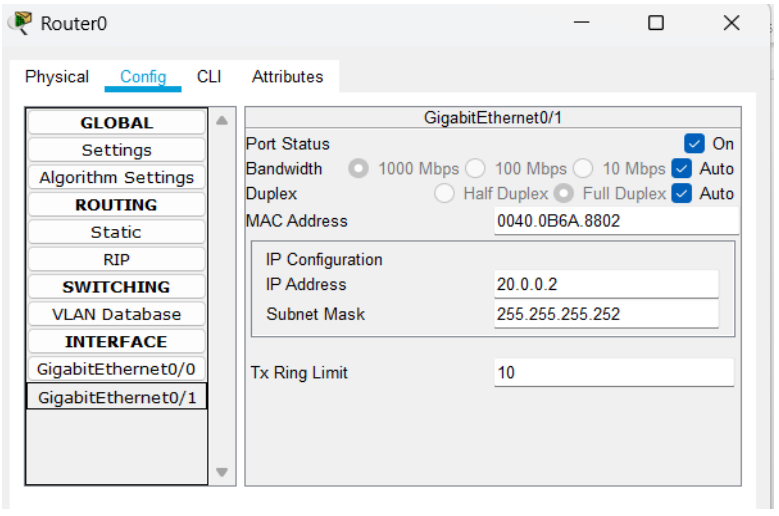
2. Configuring PC1:



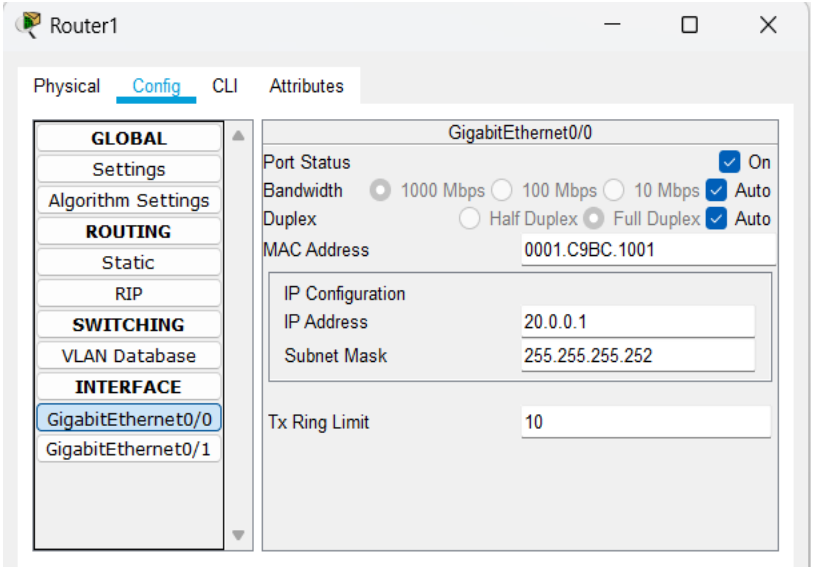
- **Configuring Router0:**
    1. **Interface GigabitEthernet0/1:**



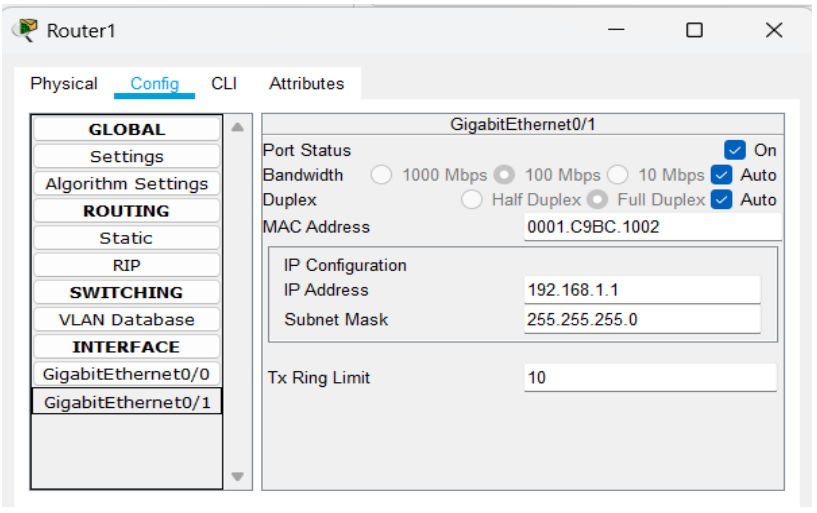
    2. **Interface GigabitEthernet0/0:**



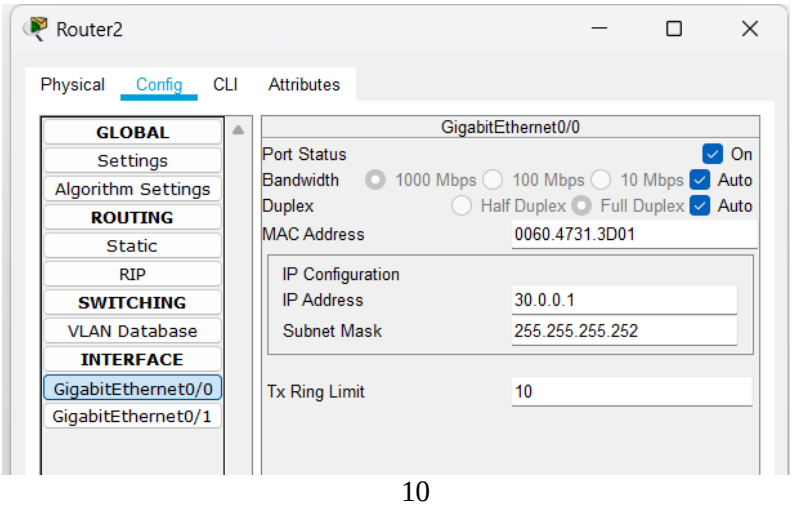
- **Configuring Router1:**
    1. **Interface GigabitEthernet0/0:**



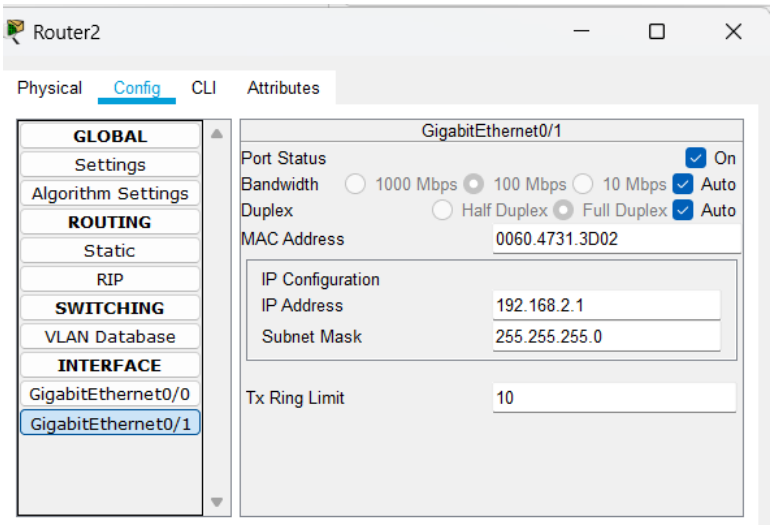
    2. **Interface GigabitEthernet0/1:**



- **Configuring Router2:**
    1. **Interface GigabitEthernet0/0:**



2. **Interface GigabitEthernet0/1:**



Checking and Enabling the Security features in Router R1 and R2:

1. **Enter the following command in the CLI mode of Router1**

Router(config)#ip route 0.0.0.0 0.0.0.0 20.0.0.2
Router(config)#hostname R1
R1(config)#exit
R1#show version

```
Device#   PID                     SN
------------------------------------------------
*0        CISCO1941/K9            FTX1524N826-


Technology Package License Information for Module:'c1900'

-----------------------------------------------------------------
Technology    Technology-package          Technology-package
              Current       Type          Next reboot
-----------------------------------------------------------------
ipbase        ipbasek9      Permanent     ipbasek9
security      None          None          None
data          None          None          None

Configuration register is 0x2102
```

(We see that the security feature is not enabled, hence we need to enable the security packageR1#)

R1#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R1(config)#
R1(config)#license boot module c1900 technology-package securityk9
R1(config)#exit
R1#
R1#copy run startup-config
R1#reload
R1>enable

R1#show version

(The security package is enabled)

```
Technology Package License Information for Module:'c1900'

-----------------------------------------------------------------
Technology     Technology-package          Technology-package
               Current        Type         Next reboot
-----------------------------------------------------------------
ipbase         ipbasek9       Permanent    ipbasek9
security       securityk9     Evaluation   securityk9
data           disable        None         None

Configuration register is 0x2102
```

2. Enter the following command in the CLI mode of Router2

Router(config)#ip route 0.0.0.0 0.0.0.0 30.0.0.2
Router(config)#hostname R2
R2(config)#exit
R2#show version

```
Device#   PID                   SN
------------------------------------------
*0        CISCO1941/K9          FTX1524N826-

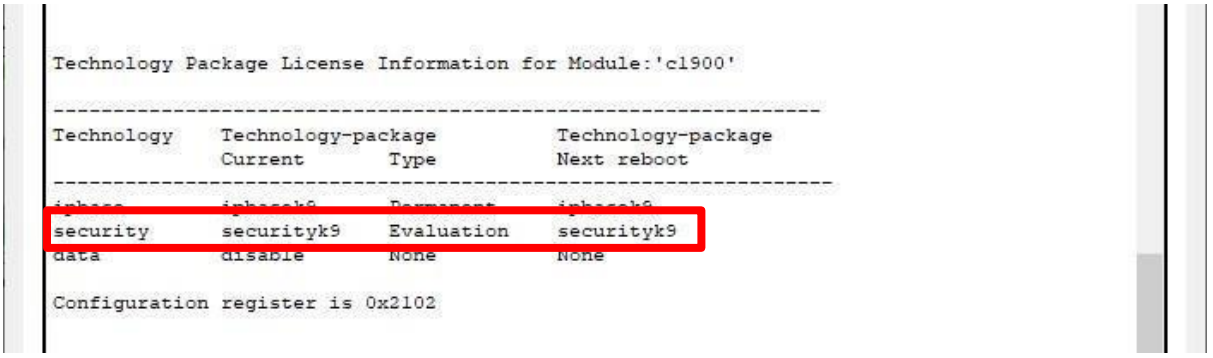

Technology Package License Information for Module:'c1900'

-----------------------------------------------------------------
Technology     Technology-package          Technology-package
               Current        Type         Next reboot
-----------------------------------------------------------------
ipbase         ipbasek9       Permanent    ipbasek9
security       None           None         None
data           None           None         None

Configuration register is 0x2102
```

(We see that the security feature is not enabled, hence we need to enable the security packageR2#)

R2#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R2(config)#
R2(config)# license boot module c1900 technology-package securityk9
R2(config)#exit
R2#
R2#copy run startup-config
R2#reload
R2>enable
R2#show version



(The security package is enabled)

Enter the following command in the CLI mode of Router0

Router>enable Router#configure terminal
Router(config)#hostname R0
R0(config)#

Defining the Hostname for all Routers and Configuring the Routers R1 and R2 for IPSec VPN tunnel

R1#configure terminal
R1(config)# access-list 100 permit ip 192.168.1.0 0.0.0.255 192.168.2.0 0.0.0.255
R1(config)#crypto isakmp policy 10
R1(config-isakmp)#encryption aes 256
R1(config-isakmp)#authentication pre-share
R1(config-isakmp)#group 5
R1(config-isakmp)#exit
R1(config)#crypto isakmp key priya address 30.0.0.1
R1(config)#crypto ipsec transform-set R1->R2 esp-aes 256 esp-sha-hmac
R1(config)#

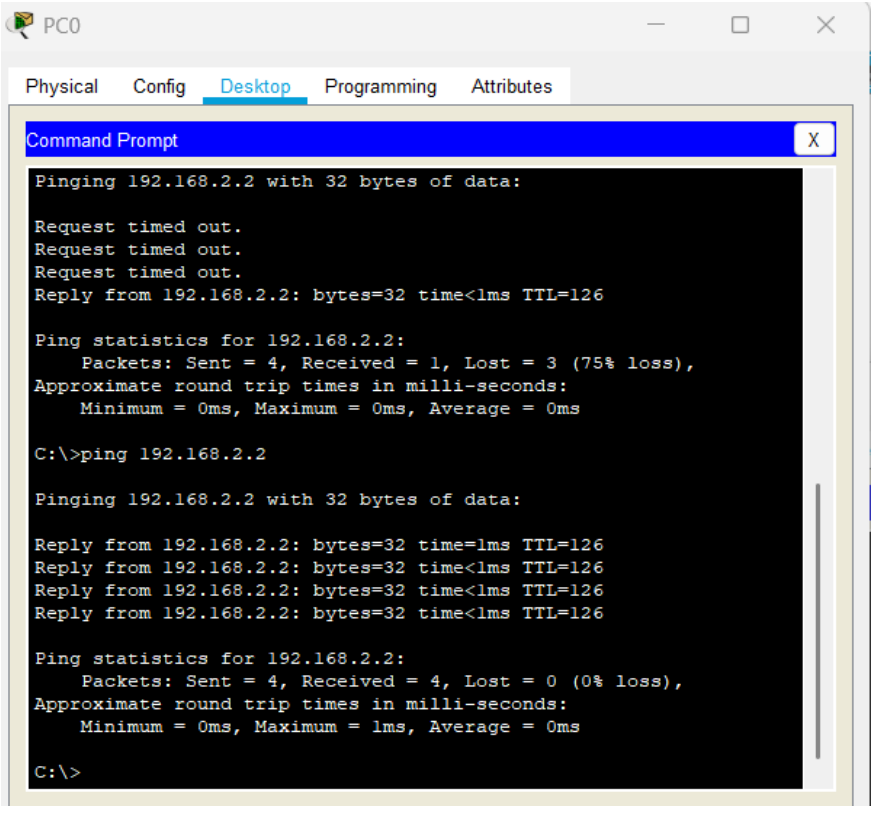
R2#
R2#configure terminal
R2(config)# access-list 100 permit ip 192.168.2.0 0.0.0.255 192.168.1.0 0.0.0.255
R2(config)#crypto isakmp policy 10
R2(config-isakmp)#encryption aes 256
R2(config-isakmp)#authentication pre-share
R2(config-isakmp)#group 5
R2(config-isakmp)#exit
R2(config)#crypto isakmp key priya address 20.0.0.1
R2(config)#crypto ipsec transform-set R2->R1 esp-aes 256 esp-sha-hmac
R2(config)#

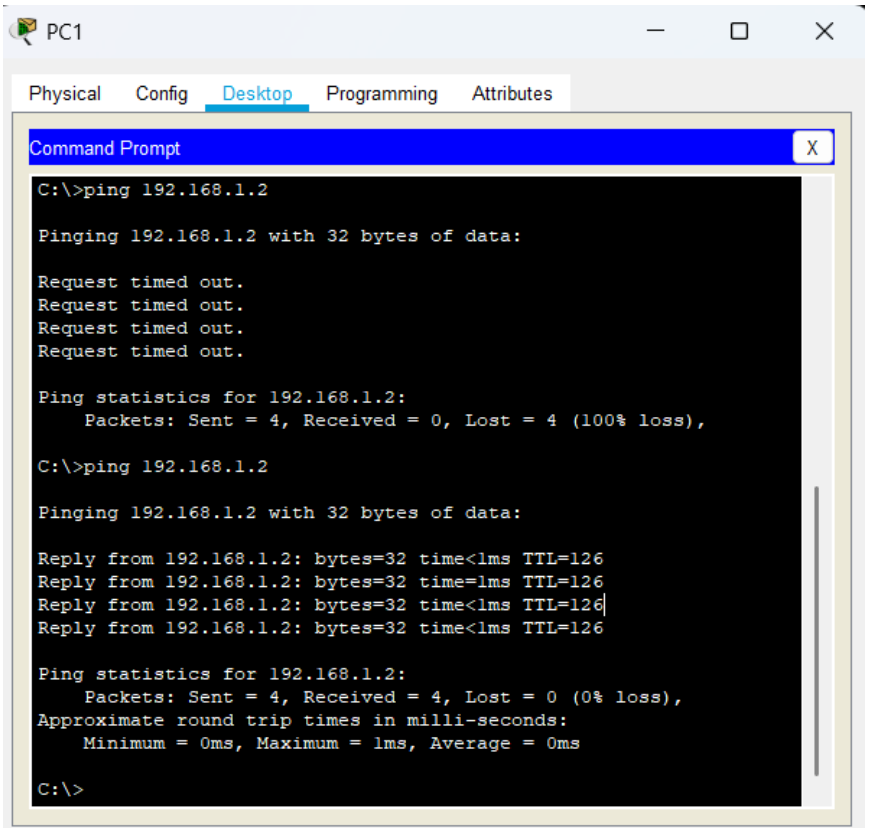
R1>enable
R1#configure terminal
R1(config)#crypto map IPSEC-MAP 10 ipsec-isakmp
R1(config-crypto-map)#set peer 30.0.0.1
R1(config-crypto-map)#set pfs group5
R1(config-crypto-map)#set security-association lifetime seconds 86400
R1(config-crypto-map)#set transform-set R1->R2
R1(config-crypto-map)#match address 100
R1(config-crypto-map)#exit
R1(config)#interface g0/0
R1(config-if)#crypto map IPSEC-MAP

R2>enable
R2#configure terminal
R2(config)#crypto map IPSEC-MAP 10 ipsec-isakmp
R2(config-crypto-map)#set peer 20.0.0.1
R2(config-crypto-map)#set pfs group5
R2(config-crypto-map)#set security-association lifetime seconds 86400
R2(config-crypto-map)#set transform-set R2->R1
R2(config-crypto-map)#match address 100
R2(config-crypto-map)#exit
R2(config)#interface g0/0
R2(config-if)#crypto map IPSEC-MAP

We verify the working of the IPSec VPN tunnel using the ping command as follows:

Output : (192.168.2.2) from PC1 and then PC1(192.168.1.2) from PC2




PC1
Physical
Config
Desktop
Programming
Attributes
Command Prompt
X
C:\>ping 192.168.1.2
Pinging 192.168.1.2 with 32 bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.
Ping statistics for 192.168.1.2:
Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
C:\>ping 192.168.1.2
Pinging 192.168.1.2 with 32 bytes of data:
Reply from 192.168.1.2: bytes=32 time<1ms TTL=126
Reply from 192.168.1.2: bytes=32 time=1ms TTL=126
Reply from 192.168.1.2: bytes=32 time<1ms TTL=126
Reply from 192.168.1.2: bytes=32 time<1ms TTL=126
Ping statistics for 192.168.1.2:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 0ms, Maximum = 1ms, Average = 0ms
C:\>

Practical No. 9

Aim: Web Security with SSL/TLS: Configure and implement secure web communication using SSL/TLS protocols, including certificate management and secure session establishment.

Client.java

```
package clientserverpractical;
import java.util.*;
import java.io.*;
import java.net.*;

public class Client {
    public static void main(String[] args) {
        Socket socket=null;
        java.io.InputStreamReader insr=null;
        java.io.OutputStreamWriter oswr=null;
        java.io.BufferedReader br=null;
        java.io.BufferedWriter bw=null;
        try {
            socket =new Socket("localhost",8081);
            insr=new InputStreamReader(socket.getInputStream());
            oswr=new OutputStreamWriter(socket.getOutputStream());
            br= new BufferedReader(insr);
            bw= new BufferedWriter(oswr);
            Scanner sc=new Scanner(System.in);
            System.out.println("Send Message to server:");
            while(true) {
                String msgtosend=sc.next();
                bw.write(msgtosend);
                bw.newLine();
                bw.flush();
                if(msgtosend.equalsIgnoreCase("BYE")) {
                    break;
                }
            }
            bw.close();
        } catch(Exception e) {
            System.out.println("Error: "+e);
        }
    }
}
```

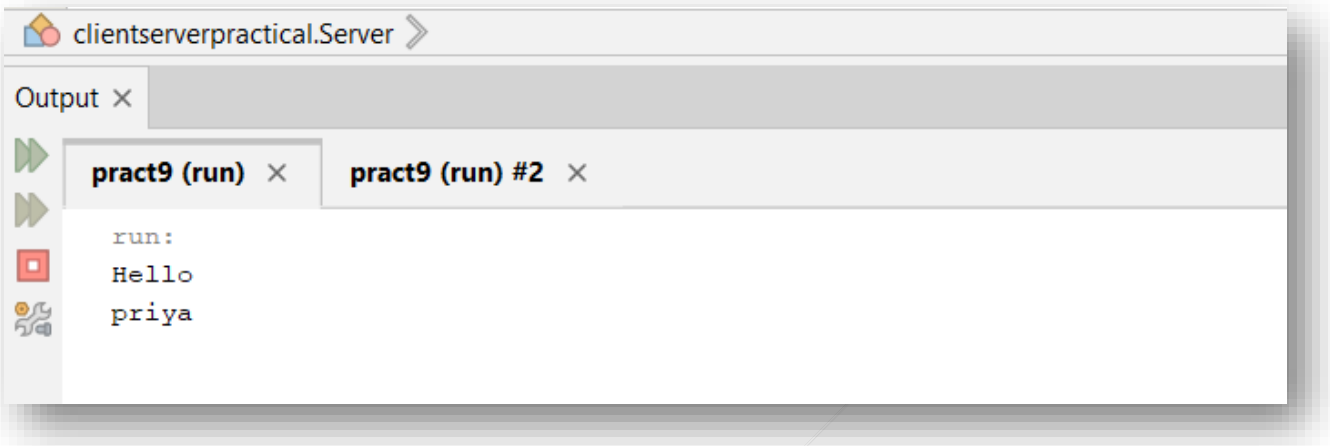
Server.java

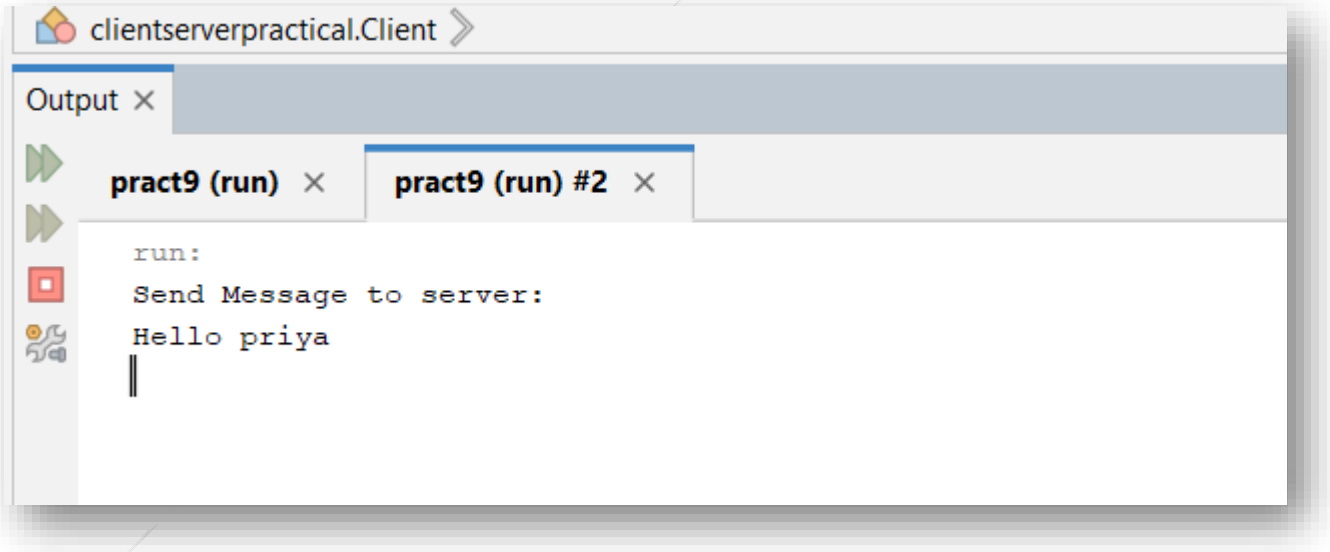
```
package clientserverpractical;
import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) throws IOException {
        Socket socket;
        ServerSocket serversocket=new ServerSocket(8081);
        java.io.InputStreamReader insr=null;
        java.io.OutputStreamWriter oswr=null;
        java.io.BufferedReader br=null;
        java.io.BufferedWriter bw=null;

        while(true) {
            try {
                socket=serversocket.accept();
                insr=new InputStreamReader(socket.getInputStream());
                oswr=new OutputStreamWriter(socket.getOutputStream());
                br= new BufferedReader(insr);
                bw= new BufferedWriter(oswr);
                while(true) {
                    String msgfromclient=br.readLine();
                    System.out.println(msgfromclient);
                    bw.write("Message Received");
                    bw.newLine();
                    bw.flush();
                    if(msgfromclient.equalsIgnoreCase("BYE")) {
                        break;
                    }
                }
                bw.close();
            } catch(Exception e) {
                System.out.println("Error: "+e);
            }
        }
    }
}
```

Output :


clientserverpractical.Server
Output
pract9 (run)
pract9 (run) #2
run:
Hello
priya


clientserverpractical.Client
Output
pract9 (run)
pract9 (run) #2
run:
Send Message to server:
Hello priya